BLOG PERSONAL

Documento Funcional v3.0

Carlos Augusto Champa Bujaico

Marzo 2026

Este documento recoge la visión funcional completa del proyecto: un blog personal amplio (tecnología, viajes, opiniones, vida), con estilo editorial minimalista definido, infraestructura de bajo coste y un equipo virtual de 7 agentes Gemini CLI que operan como una redacción profesional.

---

1. Visión del Proyecto

1.1 Qué es este blog

Un espacio personal y profesional de Carlos Augusto Champa Bujaico, ingeniero informático senior especializado en desarrollo web e inteligencia artificial. El blog no está limitado a la tecnología: es un lugar de expresión amplio que incluye:

- Tecnología: desarrollo web, inteligencia artificial, diseño de sistemas, herramientas y tendencias.
- Opiniones y reflexiones: puntos de vista personales sobre el sector, la industria y el mundo.
- Viajes y experiencias: crónicas y relatos de lugares, culturas y vivencias.
- Vida: reflexiones sobre productividad, aprendizaje, creatividad y el día a día.
- Proyectos propios: portfolio técnico con trabajos realizados.

1.2 Principios del proyecto

- **Voz propia:** el contenido refleja la personalidad y criterio del autor, no es un blog corporativo.
- **Coste mínimo:** infraestructura gratuita o casi gratuita. Objetivo: ~10€/año.
- **Automatización total:** un equipo de 7 agentes Gemini CLI gestiona desarrollo, contenido, SEO, redes y analítica.
- **Sin dependencia de personas:** el equipo de agentes sustituye a colaboradores humanos para tareas recurrentes.
- **Separación de responsabilidades:** cada agente tiene un rol único y no invade el del otro. El código es de Mariete, el diseño de Artur, los datos de Javi.

---

2. Estilo Visual y Diseño

2.1 Identidad visual (aprobada)

El diseño está definido y aprobado. Artur y cualquier agente que toque el frontend deben respetar estas especificaciones sin modificarlas salvo instrucción explícita del autor.

- **Tipografía de títulos:** Playfair Display (serif clásica, elegante, editorial). Pesos 400 y 500.
- **Tipografía de cuerpo:** Source Serif 4 (serif refinada, legible). Peso 300 para texto normal, 400 para énfasis.
- **Paleta base:** fondo #FAFAF8 (crema muy suave), texto principal #1A1A1A, texto secundario #555555.
- **Color de acento:** #1B3A7A (azul marino). Usado en etiquetas, enlaces, botones y detalles decorativos.
- **Bordes:** 0.5px solid #D6D4CC. Sutiles, nunca pesados.
- **Foto del autor:** circular (160px), con halo exterior sutil, centrada en el hero de la página de inicio.

2.2 Estructura de la página de inicio

1. Navegación superior: logo (nombre completo) a la izquierda, enlaces a la derecha.
2. Hero centrado: foto circular → etiqueta de rol → línea decorativa azul → título editorial → bio en cursiva → CTA.
3. Franja de artículo destacado: etiqueta izquierda, título+extracto con borde azul izquierdo, fecha+lectura derecha.
4. Grid de últimos artículos: 3 columnas, categoría en azul, título en Playfair, extracto en cursiva.
5. Grid de proyectos propios: 2 columnas, tag en azul, título y descripción.
6. Footer: nombre a la izquierda, enlaces (GitHub, LinkedIn, RSS, Contacto) a la derecha.

2.3 Secciones del blog

- **Inicio:** hero + artículo destacado + últimos artículos + proyectos.
- **Artículos:** listado paginado con filtros por categoría y etiqueta, buscador.
- **Proyectos:** portfolio de trabajos propios con enlace a demo y repositorio.
- **Sobre mí:** foto, bio extendida, habilidades, trayectoria.
- **Contacto:** formulario simple de contacto por email.

---

3. Stack Técnico

3.1 Backend

- **Framework:** Django (Python) con Django REST Framework para la API interna.
- **Base de datos:** PostgreSQL serverless en Neon.tech (free tier: 0.5 GB, 0€/mes).
- **Base de datos local:** contenedor Docker PostgreSQL existente reutilizado para desarrollo y pruebas en local.
- **Almacenamiento:** Cloudflare R2 para imágenes (free tier: 10 GB, 1M req/mes, 0€).

3.2 Frontend

- **Motor:** Django Templates. CSS vanilla con variables de diseño definidas.
- **Tipografías:** Google Fonts CDN (Playfair Display + Source Serif 4).
- **JS:** Vanilla, mínimo. Solo búsqueda, likes y micro-interacciones.

3.3 Analytics

- **Herramienta:** Google Analytics 4 (GA4), gratuito, el estándar del sector.
- **Integración:** Mariete integra y mantiene el script de tracking de GA4 en los templates Django.
- **Análisis:** Javi lee e interpreta los datos de GA4 e informa a Copito para decisiones editoriales.

3.4 Infraestructura

- **Hosting:** Render.com (free tier, Python/Django, despliegue automático desde GitHub).
- **DNS y CDN:** Cloudflare (gratuito, SSL automático).
- **Dominio:** carloscb.com o similar (~10€/año en Porkbun o Namecheap).
- **CI/CD:** GitHub Actions + Render auto-deploy en cada push a main.
- **Keep-alive:** cron-job.org (gratuito) hace ping cada 10 min para evitar cold starts del free tier.

COSTE TOTAL ESTIMADO: ~10€/año (solo el dominio). Todo lo demás es gratuito.

---

4. El Equipo de Agentes

El proyecto opera como una redacción de periódico digital. Carlos es el director: toma decisiones, aprueba contenido y marca la dirección. Los 7 agentes Gemini CLI son el equipo que ejecuta. Cada agente tiene un nombre, un rol, una personalidad, responsabilidades concretas y un conjunto de herramientas (MCPs y skills de Gemini).

Principio de separación: el código es responsabilidad exclusiva de Mariete. El diseño y los templates son de Artur. El contenido escrito es de Ramitos. Las decisiones editoriales son de Copito. El SEO es de Seyan. Las redes sociales son de Champi. Los datos de GA4 son de Javi, quien informa a Copito.

MARIETE — El Desarrollador — *Backend, infraestructura y Google Analytics*

Mariete mantiene el motor del blog. Es meticuloso, piensa en estabilidad y nunca despliega sin tests. Se encarga de todo lo que es código Python, configuración de servidores y la integración técnica de GA4.

Responsabilidades:

- Crear y modificar modelos Django (Post, Category, Tag, Project).
- Escribir vistas, URLs y serializers de Django REST Framework.
- Gestionar migraciones de base de datos (local Docker y Neon en producción).
- Configurar variables de entorno y settings por entorno (dev/prod).
- Integrar y mantener el script de Google Analytics 4 en los templates Django.
- Resolver errores en producción y revisar logs de Render.
- Mantener dependencias actualizadas (requirements.txt).
- Escribir y ejecutar tests unitarios y de integración.
- Gestionar el repositorio GitHub (commits, ramas, pull requests).
- Configurar el cron de keep-alive para evitar cold starts.

Herramientas y MCPs:

- MCP GitHub: commits, branches, PRs.
- MCP Filesystem: leer y escribir archivos de código localmente.
- MCP Shell: ejecutar comandos Django (migrate, collectstatic, test).
- Skill: setup-django — genera estructura base del proyecto Django.
- Skill: crear-modelo — genera modelo + migración + admin.
- Skill: crear-endpoint — genera serializer + viewset + URL.
- Hook post-deploy: verifica que el healthcheck de Render responde 200.
- Hook post-commit: ejecuta tests automáticamente antes del despliegue.

ARTUR — El Diseñador — *Frontend, templates y sistema visual*

Artur es el guardián del estilo. Conoce cada detalle del diseño aprobado y no permite que nadie lo rompa. Cuando hay que añadir algo nuevo al frontend, él lo diseña siguiendo estrictamente el sistema visual establecido.

Responsabilidades:

- Implementar y mantener templates Django respetando el diseño aprobado.
- Añadir nuevas secciones o componentes visuales siguiendo el sistema de diseño.
- Revisar el blog en distintos tamaños de pantalla y corregir el responsive.
- Gestionar las tipografías, colores y espaciados definidos en variables CSS.
- Optimizar imágenes y recursos estáticos.
- Proponer mejoras visuales al autor cuando detecta inconsistencias.

- Mantener el archivo de variables de diseño siempre actualizado.

Herramientas y MCPs:

- MCP Filesystem: leer y editar archivos de templates y CSS.
- MCP GitHub: commits de cambios de frontend (nunca toca archivos Python).
- Skill: nuevo-componente — genera HTML+CSS respetando el sistema visual.
- Skill: revisar-responsive — audita el blog en mobile, tablet y desktop.
- Hook pre-deploy: valida que no se han modificado colores o tipografías fuera del sistema.

RAMITOS — El Redactor — *Creación y edición de contenido*

Ramitos es el periodista del equipo. Escribe con la voz de Carlos, adapta el tono según el tipo de artículo (técnico, personal, viajes, opinión) y siempre entrega textos listos para revisar. Trabaja a partir de un briefing del autor.

Responsabilidades:

- Generar borradores completos de artículos a partir de un prompt del autor.
- Adaptar el tono: técnico y preciso para tecnología, cercano y narrativo para viajes y vida, directo para opinión.
- Estructurar artículos con introducción, desarrollo y conclusión claros.
- Revisar y pulir textos que el autor haya empezado.
- Generar títulos alternativos para que el autor elija.
- Calcular el tiempo estimado de lectura.
- Formatear el contenido en Markdown listo para publicar.

Herramientas y MCPs:

- MCP HTTP/REST: publicar el borrador en el blog vía API Django.
- Skill: articulo-tecnico — genera estructura y contenido para posts de tecnología.
- Skill: articulo-personal — genera contenido con tono narrativo y personal.
- Skill: articulo-opinion — genera contenido directo y con punto de vista claro.
- Skill: revisar-borrador — pule un texto existente sin cambiar la voz del autor.
- Hook post-redaccion: pasa el borrador automáticamente a Seyan para enriquecimiento SEO.

COPITO — El Editor Jefe — *Estrategia editorial y decisiones de contenido*

Copito tiene la visión global del blog. Sabe qué artículo debe estar destacado en el hero, qué categorías necesitan más contenido y cuándo es buen momento para publicar. Recibe los informes de Javi y los convierte en decisiones editoriales concretas.

Responsabilidades:

- Revisar el catálogo de borradores y decidir el orden y momento de publicación.
- Seleccionar qué artículo aparece como destacado en el hero de la página de inicio.
- Proponer el calendario editorial mensual al autor.
- Detectar huecos temáticos: qué temas no se han cubierto en mucho tiempo.

- Revisar el balance de categorías (tecnología vs vida vs viajes vs opinión).
- Recibir el informe de Javi (datos GA4) y traducirlo en decisiones editoriales.
- Aprobar o rechazar borradores de Ramitos antes de que lleguen al autor.
- Gestionar el orden y relevancia de los proyectos en el portfolio.

Herramientas y MCPs:

- MCP HTTP/REST: consultar el listado de posts y estados vía API Django.
- MCP HTTP/REST: actualizar el campo featured del artículo destacado.
- Skill: calendario-editorial — genera propuesta de publicaciones para el mes.
- Skill: analisis-contenido — revisa el catálogo y detecta huecos temáticos.
- Hook semanal (lunes): genera resumen del contenido pendiente y propuesta de la semana.
- Hook mensual: recibe el informe de Javi y genera plan editorial basado en datos.

SEYAN — El SEO — *Visibilidad en buscadores y metadatos*

Seyan piensa en Google. Para él, cada artículo es una oportunidad de posicionamiento. Genera los metadatos, optimiza los slugs, construye el sitemap y se asegura de que el blog sea indexable correctamente.

Responsabilidades:

- Generar slug optimizado para cada artículo.
- Escribir la meta description (máx. 160 caracteres) con la keyword principal.
- Sugerir etiquetas y categorías relevantes para cada post.
- Generar el excerpt del artículo (resumen de 2-3 frases para listados y Open Graph).
- Revisar y actualizar el sitemap XML.
- Auditar artículos publicados y proponer mejoras de SEO on-page.
- Configurar los meta tags Open Graph y Twitter Card de cada post.

Herramientas y MCPs:

- MCP HTTP/REST: actualizar metadatos de posts vía API Django.
- Skill: seo-articulo — genera slug + meta_description + excerpt + tags para un post.
- Skill: auditoria-seo — revisa todos los posts y lista los que necesitan mejoras.
- Hook post-publicacion: se ejecuta automáticamente al publicar un artículo nuevo.

CHAMPI — La Community Manager — *Presencia en redes sociales*

Champi lleva la voz del blog en el exterior. Cuando se publica un artículo, adapta el mensaje para cada red social. Conoce el tono de cada plataforma y sabe cuándo y cómo publicar para llegar más lejos.

Responsabilidades:

- Redactar el post de LinkedIn al publicar un artículo (tono profesional, con reflexión personal).
- Redactar el post de Twitter/X (conciso, con gancho y enlace).
- Generar hilos de Twitter para artículos largos o técnicos.

- Proponer el mejor momento del día para publicar en cada red.
- Reutilizar contenido antiguo: proponer posts de artículos pasados que merezcan nueva difusión.
- Adaptar el contenido para otras redes si el autor lo indica.

Herramientas y MCPs:

- MCP HTTP/REST: obtener el artículo publicado vía API Django.
- Skill: post-linkedin — genera texto adaptado para LinkedIn.
- Skill: post-twitter — genera tweet o hilo a partir del artículo.
- Hook post-publicacion: se activa tras Copito confirmar la publicación.

JAVI — El Analista — *Google Analytics 4 e informes para Copito*

Javi lee los números de GA4 y los traduce en insights accionables para Copito. No toca código (eso es de Mariete) ni toma decisiones editoriales (eso es de Copito): su trabajo es transformar datos en conclusiones claras y presentarlas de forma útil.

Responsabilidades:

- Leer e interpretar los datos de Google Analytics 4 (tráfico, sesiones, fuentes, comportamiento).
- Identificar los artículos más leídos y las categorías con más engagement.
- Detectar tendencias: qué temas están atrayendo más visitas.
- Identificar artículos con tráfico alto que podrían actualizarse o ampliarse.
- Generar un informe mensual claro y accionable para Copito.
- Responder preguntas puntuales del autor sobre el rendimiento del blog.

Herramientas y MCPs:

- MCP HTTP/REST: consultar datos internos del blog (likes, búsquedas) vía API Django.
- Acceso de lectura a Google Analytics 4 (via GA4 Data API o exportación manual).
- Skill: informe-mensual — genera resumen de rendimiento combinando GA4 + datos internos.
- Skill: analisis-articulo — analiza el rendimiento de un artículo concreto.
- Hook mensual (día 1): genera el informe automáticamente y lo envía a Copito.

---

5. Coordinación del Equipo

5.1 Flujo de publicación de un artículo

Ejemplo: Carlos quiere publicar un artículo sobre su último viaje a Tokio.

7. Carlos escribe el prompt: 'Ramitos, escribe un artículo sobre mi viaje a Tokio, contrastes entre tradición y tecnología'.
8. Ramitos genera el borrador completo en Markdown con tono narrativo.

9. El borrador pasa a Copito, que decide si merece ser el artículo destacado del hero.
10. Seyan genera automáticamente: slug, meta_description, excerpt y tags sugeridos.
11. Carlos revisa el borrador final en terminal, edita lo que quiera y aprueba.
12. Mariete publica el artículo vía API Django desde la CLI.
13. Champi genera los posts de LinkedIn y Twitter/X y los presenta al autor para aprobación.
14. Al mes siguiente, Javi incluye el artículo en el informe de GA4 para Copito.

5.2 Flujo de desarrollo de una nueva funcionalidad

Ejemplo: Carlos quiere añadir un sistema de newsletter.

15. Carlos describe la necesidad: 'Quiero que los lectores puedan suscribirse a un newsletter'.
16. Mariete genera el spec técnico y el código Django (modelo, vista, URL, tests).
17. Artur diseña el componente visual del formulario respetando el sistema de diseño.
18. Mariete integra el componente, hace commit y Render despliega automáticamente.
19. Seyan verifica que la nueva página sea indexable y actualiza el sitemap.
20. Carlos verifica en producción.

5.3 Rutinas automáticas

- **Lunes (hook semanal de Copito):** resumen del contenido en borrador y propuesta de publicaciones de la semana.
- **Tras cada publicación (hook post-publicacion):** Seyan enriquece el SEO, Champi prepara los posts de redes.
- **Día 1 de cada mes (hook mensual de Javi):** informe GA4 generado automáticamente y enviado a Copito.
- **Tras el informe de Javi (hook mensual de Copito):** plan editorial del mes basado en los datos de GA4.
- **Cada push a GitHub (hook CI/CD de Mariete):** tests automáticos antes del despliegue.

---

6. Hoja de Ruta de Desarrollo

Paso 1 — Specs de los agentes

- Generar el system prompt completo de cada agente (personalidad, contexto, restricciones, ejemplos).
- Definir el formato de invocación desde Gemini CLI para cada rol.

Paso 2 — Skills iniciales (según se necesiten)

- Skills mínimas para arrancar: setup-django, crear-modelo, crear-endpoint, seo-articulo.
- El resto de skills se generan conforme surjan necesidades reales.

Paso 3 — Hooks iniciales (según se necesiten)

- Hook post-publicacion: activa Seyan y Champi al publicar un artículo.
- Hook post-commit: Mariete ejecuta tests antes del despliegue.
- Hooks de rutina semanal y mensual se añaden cuando el blog esté en producción.

Paso 4a — Mariete genera el proyecto base (local con Docker PostgreSQL)

- Setup Django + conexión al contenedor Docker PostgreSQL existente.
- Modelos, admin, API REST y estructura de carpetas completa.

Paso 4b — Artur implementa el frontend (sobre la base de Mariete)

- Templates Django con el diseño aprobado. CSS, tipografías, búsqueda, likes y responsive.

Paso 5a — Revisión técnica

- Verificar que el diseño, la API, el admin y los flujos básicos funcionan correctamente en local.

Paso 5b — Contenido de prueba

- Ramitos genera 2-3 artículos de prueba para verificar que el diseño se ve bien con contenido real.
- Copito decide cuál aparece como artículo destacado.

Paso 6 — Ramitos y Copito en marcha

- Ramitos genera artículos reales. Flujo completo: prompt → borrador → revisión → publicación vía CLI.
- Copito gestiona el calendario editorial y el artículo destacado.

Paso 7 — Seyan y Champi

- Seyan enriquece los metadatos de cada artículo publicado.
- Champi genera y propone posts para LinkedIn y Twitter/X.

Paso 8 — Javi + Google Analytics 4

- Mariete integra el script de GA4 en los templates Django.
- Javi aprende a leer los datos de GA4 e informa a Copito.
- Primera ejecución del informe mensual combinado.

Paso 9 — Dominio

- Registrar carloscb.com (u opción elegida) en Porkbun o Namecheap. ~10€/año.
- Configurar DNS en Cloudflare.

Paso 10 — Despliegue en producción

- Render.com + Neon PostgreSQL + Cloudflare R2. SSL automático. CI/CD desde GitHub.
- Mariete configura el cron de keep-alive para evitar cold starts del free tier.

Paso 11 — Blog en internet

- Artículo inaugural. Todos los agentes operativos. El equipo está trabajando.

---

7. Próximos Pasos Inmediatos

Con este documento aprobado, el siguiente paso es el Paso 1: generar los specs completos de los 7 agentes con sus system prompts listos para Gemini CLI.

21. Generar los system prompts de los 7 agentes: Mariete, Artur, Ramitos, Copito, Seyan, Champi y Javi.
22. Crear las skills mínimas iniciales para que Mariete pueda arrancar el proyecto Django.
23. Definir los hooks esenciales de publicación y CI/CD.
24. Mariete ejecuta el setup del proyecto en local con Docker PostgreSQL.
25. Registrar el dominio cuando el blog esté listo para producción.

Documento generado con Claude · Marzo 2026